# 3.4 Multiple input and multiple output channels

- while we have described the multiple channels that comprise each image (e.g., color images have the standard RGB channels to indicate the amount of red, green, and blue) and convolutional layers for multiple channels in Section 3.1, until now, we simplified all of our numerical examples by working with just a single input and a single output channel
- this has allowed us to think of our inputs, convolution kernels, and outputs each as two-dimensional tensors
- when we add channels into the mix, our inputs and hidden representations both become three-dimensional tensors; for example, each RGB input image has shape $3 \times h \times w$
- we refer to this axis, with a size of 3, as the *channel dimension*
- in this section, we will take a deeper look at convolution kernels with multiple input and multiple output channels

# 3.4 Multiple input and multiple output channels

- when the input data contain multiple channels, we need to construct a convolution kernel with the same number of input channels as the input data, so that it can perform cross-correlation with the input data
- assuming that the number of channels for the input data is $c_{in}$, the number of input channels of the convolution kernel also needs to be $c_{in}$
- if our convolution kernel's window shape is $k_h \times k_w$, then, when $c_{in} = 1$, we can think of our convolution kernel as just a two-dimensional tensor of shape $k_h \times k_w$
- however, when $c_{in} > 1$, we need a kernel that contains a tensor of shape $k_h \times k_w$ for every input channel
- concatenating these $c_{in}$ tensors together yields a convolution kernel of shape $c_{in} \times k_h \times k_w$

# 3.4 Multiple input and multiple output channels

- since the input and convolution kernel each have $c_{in}$ channels, we can perform a cross-correlation operation on the two-dimensional tensor of the input and the two-dimensional tensor of the convolution kernel for each channel, adding the $c_{in}$ results together (summing over the channels) to yield a two-dimensional tensor
- this is the result of a two-dimensional cross-correlation between a multi-channel input and a multi-input-channel convolution kernel
- in Figure 6, we demonstrate an example of a two-dimensional cross-correlation with two input channels
- the shaded portions are the first output element, as well as the input and kernel tensor elements used for the output computation:
$(1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4) + (0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3) = 56.$

# 3.4 Multiple input and multiple output channels

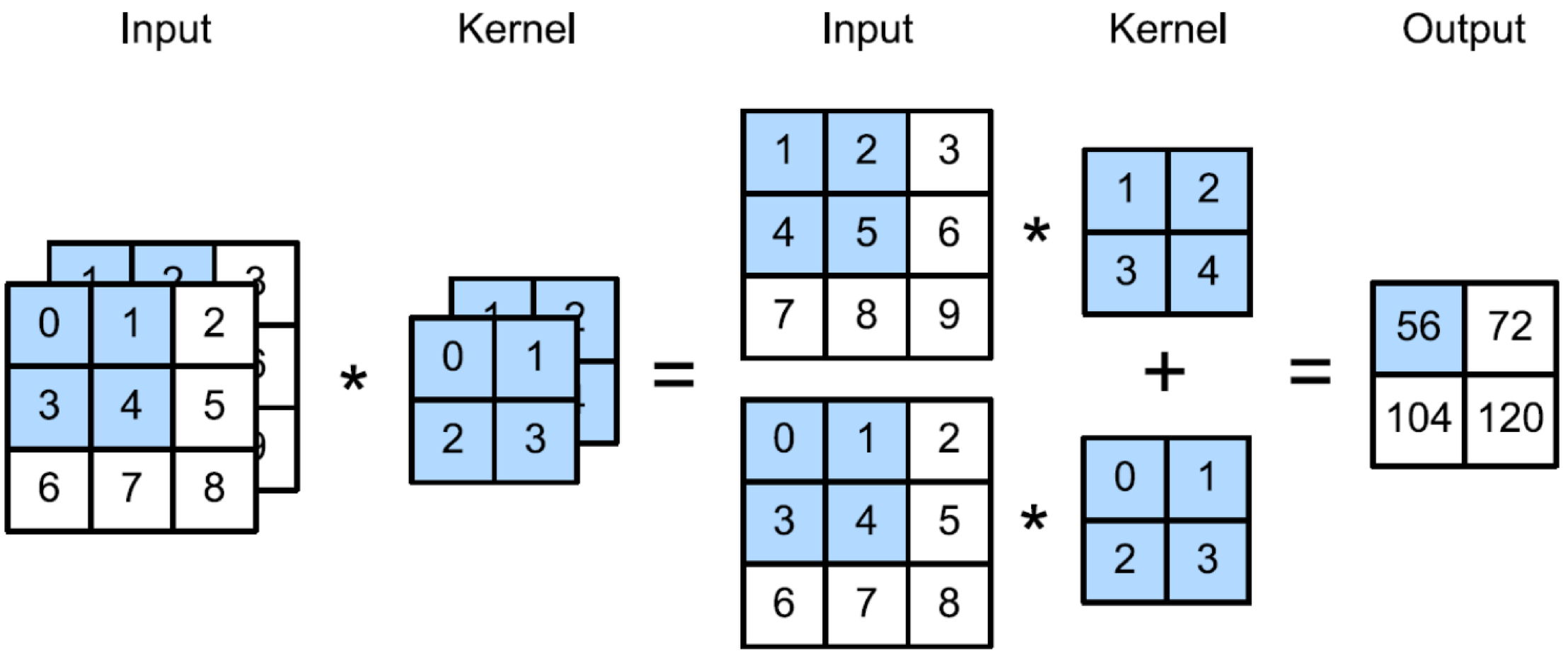


Figure 6: Cross-correlation computation with 2 input channels.

# 3.4 Multiple input and multiple output channels

- regardless of the number of input channels, so far we always ended up with one output channel
- however, as we discussed in Section 3.1, it turns out to be essential to have multiple channels at each layer
- in the most popular neural network architectures, we actually increase the channel dimension as we go higher up in the neural network, typically downsampling to trade off spatial resolution for greater *channel depth*
- intuitively, we could think of each channel as responding to some different set of features
- reality is a bit more complicated than the most naive interpretations of this intuition, since representations are not learned independently, but are rather optimized to be jointly useful
- so it may not be that a single channel learns an edge detector, but rather that some direction in channel space corresponds to detecting edges

# 3.4 Multiple input and multiple output channels

- denote by $c_{in}$ and $c_{out}$ the number of input and output channels, respectively, and let $k_h$ and $k_w$ be the height and width of the kernel
- to get an output with multiple channels, we can create a kernel tensor of shape $c_{in} \times k_h \times k_w$ for *every* output channel
- we concatenate them on the output channel dimension, so that the shape of the convolution kernel is $c_{out} \times c_{in} \times k_h \times k_w$
- in cross-correlation operations, the result on each output channel is calculated from the convolution kernel corresponding to that output channel, and takes input from all channels in the input tensor

# 3.4 Multiple input and multiple output channels

- at first, a $1 \times 1$ convolution, i.e., $k_h = k_w = 1$, does not seem to make much sense
- after all, a convolution correlates adjacent pixels; a $1 \times 1$ convolution obviously does not
- nonetheless, they are popular operations that are sometimes included in the designs of complex deep networks
- because the minimum window is used, the $1 \times 1$ convolution loses the ability of larger convolutional layers to recognize patterns consisting of interactions among adjacent elements in the height and width dimensions
- the only computation of the $1 \times 1$ convolution occurs on the channel dimension

# 3.4 Multiple input and multiple output channels

- Figure 7 shows the cross-correlation computation using the $1 \times 1$ convolution kernel with 3 input channels and 2 output channels
- note that the inputs and outputs have the same height and width
- each element in the output is derived from a linear combination of elements *at the same position* in the input image
- we could think of the $1 \times 1$ convolutional layer as constituting a fully-connected layer applied at every single pixel location to transform the $c_{in}$ corresponding input values into $c_{out}$ output values
- because this is still a convolutional layer, the weights are tied across pixel location
- thus, the $1 \times 1$ convolutional layer requires $c_{out} \times c_{in}$ weights (plus the bias)

# 3.4 Multiple input and multiple output channels

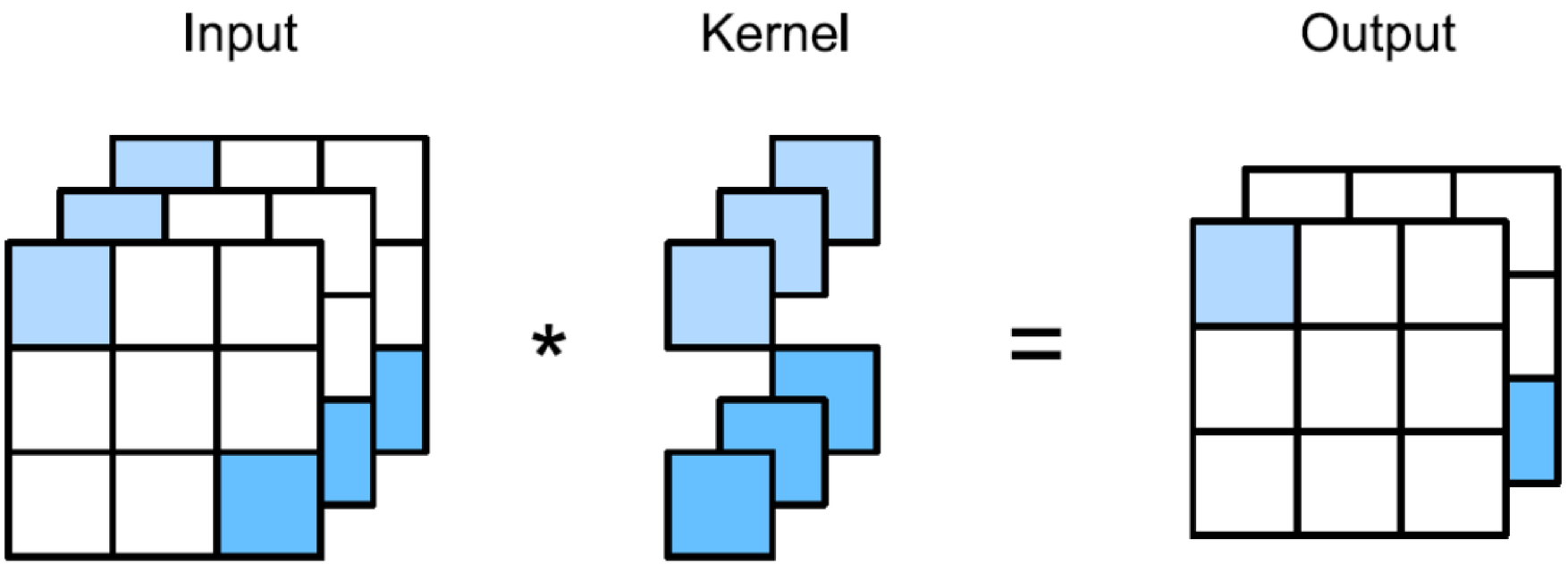


Figure 7: The cross-correlation computation uses the $1 \times 1$ convolution kernel with 3 input channels and 2 output channels. The input and output have the same height and width.